



Technical Document

Architecture, AI Review & System Guide



Multi-Mind | Team Code X898L7U7

Shu Zixuan • Yao Xinyan • Hu Chengcheng • Liu Mingda

Contents

1 Executive Technical Summary 1

2 Scope, Sources, and Quality Constraints 1

2.1 Document Scope 1

2.2 Users and Non-Functional Requirements 1

3 System Architecture 1

3.1 System Context and Deployment Topology 1

3.2 Frontend Architecture 2

3.3 Backend Architecture 2

3.4 AI Agent Architecture 2

3.5 Storage, State, and Audit 3

3.6 Security and Failure Boundaries 3

4 Core Workflows 4

4.1 Request Creation and Client Submission 4

4.2 Financial Evidence Rules 4

5 Product Capabilities 4

5.1 Role-Aligned Workspaces 4

5.2 Functional Modules 6

5.3 AI Modes and Their Boundaries 6

6 System Usage Guide 7

6.1 Getting Started and Reading Status 7

6.2 Firm Administrator: Set Up a Client 7

6.3 Accountant: Create, Check, and Publish a Request 8

6.4 Client: Find the Request and Interpret the Checklist 9

6.5 Client: Upload and Place Documents 10

6.6 Client: Submit, Correct, or Challenge a Result 10

6.7 Accountant: Review Evidence and Record Decisions 11

6.8 Worked AI Review: Reimbursement Evidence 13

6.9 Contacts, Notifications, and Personal Settings 14

6.10 Common Blocking Conditions 14

7 API Documentation 15

7.1 Public Backend Contract 15

7.2 Authentication and Error Handling 15

7.3	Primary Resource Operations	15
7.4	Representative Request Contracts	16
7.5	Private Backend–Agent Contract	16
8	Folio-Reasoner-8B and Model Evaluation	17
8.1	Model Role and Provider Boundary	17
8.2	Accounting Dataset and Executable Environment	17
8.3	Training Procedure	18
8.4	FolioBench-120 Results and Interpretation	18
9	Technical Report: Operational Results	19
9.1	Completion and Answer-Quality Rates	19
9.2	Illustrative Review Findings	20
9.3	Interpretation and Next Verification Gate	20
10	Deployment and Operations	20
10.1	Build and Release Topology	20
10.2	Production Configuration	20
10.3	Controlled Deployment Procedure	22
10.4	Rollback, Backup, and Known Gaps	22
A	Source Register and Measurement Notes	22
B	Complete Backend Endpoint Inventory	22

1 Executive Technical Summary

Folio is a document-collection and evidence-review system for accounting firms and their clients. A Vue single-page application presents separate staff and client workspaces. FastAPI owns authentication, tenant authorization, request state, audit history, and all business writes. A database-lease Worker scans files and orchestrates asynchronous classification and review. A separate, read-only Agent performs document OCR and structured model analysis; the Backend validates its output and remains the authority for search, monetary checks, and state transitions. Folio-Reasoner-8B is the selected specialist for accounting reasoning; the model-service boundary also supports other engines through compatible adapters. The accountant alone confirms the overall request.

The deployed demonstration uses Docker Compose on an AWS Lightsail host in Singapore behind Nginx and Cloudflare. PostgreSQL is the system of record and the source of Worker leases; Redis stores revocable session and rate-limit state, not the task queue. The file boundary is a quarantine volume followed by ClamAV scanning and controlled promotion to a documents volume. OCR and accounting reasoning are separate model services, so the selected reasoning model can change without moving business authority out of the Backend. Section 8 reports Folio-Reasoner-8B's controlled synthetic evaluation; Section 9 separately reports operational results from the demonstration deployment.

2 Scope, Sources, and Quality Constraints

2.1 Document Scope

This document covers the delivered request-to-review workflow, its interfaces, the demonstration deployment, the selected reasoning model, and operational results. It does not describe general-ledger posting, tax filing, payments, bank account operation, or a publicly exposed Agent API. Architecture follows the system design specification; endpoint names were checked against the Backend OpenAPI schema and Agent routes. Operational results are separated from design intent and model-level evaluation.

2.2 Users and Non-Functional Requirements

The fixed roles are firm administrator, accountant, client administrator, and client submitter. The Backend enforces firm membership, client assignment, and role on every request; frontend route guards are not a security boundary. A usable evidence record must retain the requirement, submission round, source documents, review findings, linked evidence, user-visible follow-up, and human decision. A model outage must leave the manual upload and review path available. File content and model output cross explicit trust boundaries, so scanning, schema validation, authorization, and deterministic monetary checks are mandatory.

3 System Architecture

3.1 System Context and Deployment Topology

Browsers reach the HTTPS Nginx entry point, which serves the static frontend and proxies `/api/v1` to FastAPI. The Backend and Worker share an image but run separate processes. The Worker calls the Agent over the internal Compose network. The Agent mounts the approved documents volume read-only, obtains page text through an OCR service, and sends the text with controlled task context to the configured reasoning model. No direct database, Redis, JWT secret, public host port, or business-write capability is granted to the Agent. Email and Feishu delivery are disabled; in-app notifications work.

3.2 Frontend Architecture

Figure 1 condenses the current frontend architecture drawing: app shell, router, staff and portal branches, feature API modules, shared UI, session preferences, and the Vite-to-static delivery path. Business data is fetched from the server; Pinia holds authentication and UI preferences. The Axios boundary converts API snake_case to frontend camelCase. English and Simplified Chinese are supported in the interface.

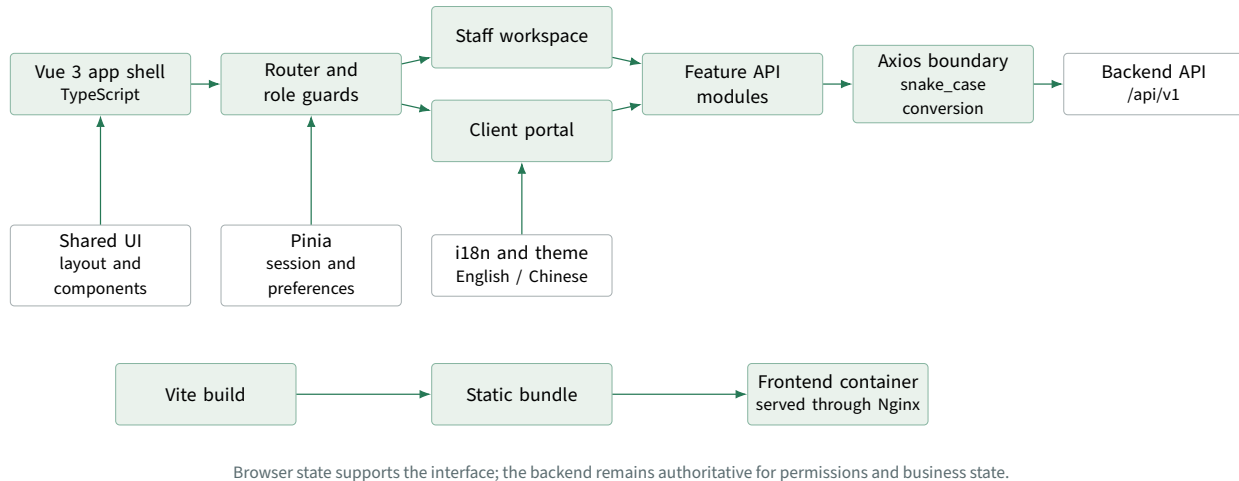


Figure 1: Frontend architecture, simplified and translated from the current frontend framework diagram.

3.3 Backend Architecture

Figure 2 preserves the current backend drawing's control path: ingress, authorization and business services, database transaction, lease-driven Worker, file scan, scoped search, Agent call, and output validation. The Worker does not use Redis as a queue. A successful Agent HTTP call is not sufficient for an automatic business decision; the Backend checks identifiers, ownership, evidence, amount relations, and allowed state changes first.

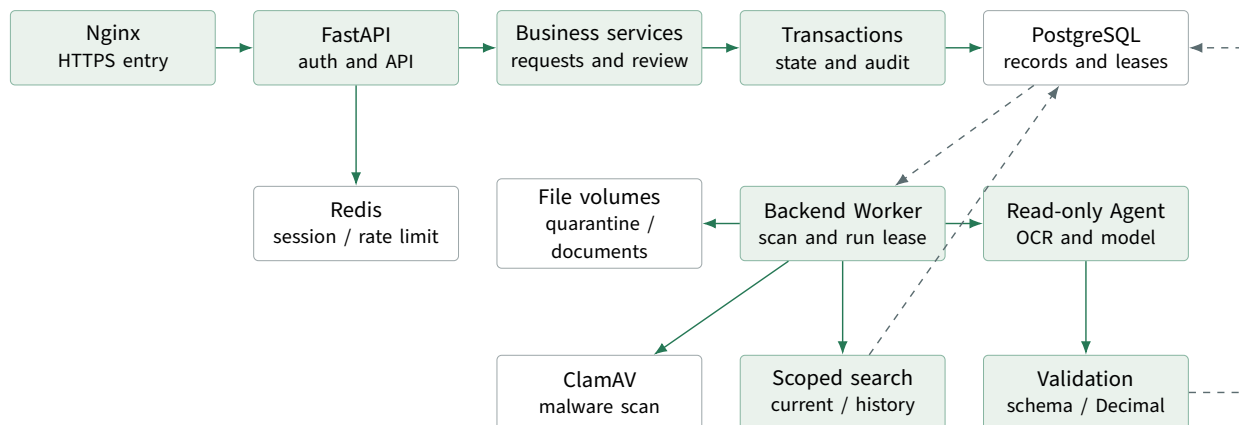
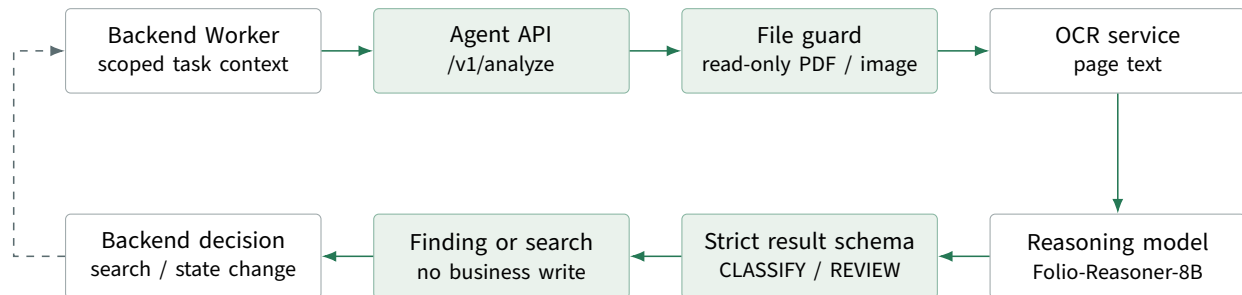


Figure 2: Backend architecture, simplified and translated from the current backend framework diagram.

3.4 AI Agent Architecture

The Agent has two purposes. CLASSIFY runs after upload and safety scanning, returning category, document type, target requirement, and confidence only. It must not judge entity, period, amount, or business state.

REVIEW runs only after formal client submission and returns extractions, findings, evidence, monetary relations, or a bounded search request. The Backend performs `SEARCH_CURRENT` and `SEARCH_HISTORY` within the authorized client scope and sends the result in a subsequent Agent turn. In the OCR-to-reasoner path, the reasoning model receives text and controlled context rather than raw file bytes or filenames. Folio-Reasoner-8B is the selected specialist; another model must implement the same task schema and pass Backend validation.



A search request returns to the Backend; the next Agent turn receives only authorized search results.

Figure 3: Agent analysis path and the Backend-owned search loop.

3.5 Storage, State, and Audit

Component	Responsibility
PostgreSQL	Firms, users, clients, requests, requirements, submissions, document metadata, immutable review decisions, workflow events, AI runs, notifications, and lease timestamps.
Redis	Refresh-session and revocation data, rate limits, and other reconstructible state.
File volumes	Quarantine for untrusted uploads; approved documents after a clean scan; Nginx and Agent receive only the controlled read-only view they require.
Outbox	External notification records are retained, but provider delivery remains suppressed.

The request states include `DRAFT`, `OPEN`, `IN_REVIEW`, `CHANGES_REQUESTED`, `READY_FOR_BOOKKEEPING`, and `CANCELLED`. Requirement states include `PENDING`, `RECEIVED`, `NEEDS_ACTION`, `SATISFIED`, and `WAIVED`. State changes use action endpoints and optimistic `version` checks rather than arbitrary status updates. Each correction is a new submission round; old files and decisions remain available for audit. An item may be automatically satisfied or returned when evidence passes Backend guardrails, but whole-request confirmation is always human.

3.6 Security and Failure Boundaries

Uploads accept PDF, PNG, and JPEG subject to size, declared MIME, file-header, and SHA-256 checks. The Worker uses ClamAV in production before moving a file from quarantine to documents. Downloads are authorized by Backend and served through a protected Nginx redirect. The Agent checks storage keys, regular-file properties, size, MIME signature, and hash before read. Model errors and exhausted retries leave the request for human review rather than silently changing accounting state. Application logs identify requests and runs; audit events and immutable decisions, not log lines, are the durable business record.

4 Core Workflows

4.1 Request Creation and Client Submission

1. Staff select a client, accounting period, due date, assignee, AI mode, and review preference; they write client-facing instructions and add typed requirements. A request is saved as a draft, then explicitly published.
2. A client opens the portal, uploads PDF or image files, and may use quick classification. Each file remains in staging until the client confirms its destination requirement or marks it as other/invalid. Failed classification has a manual placement path.
3. After confirmation, the client submits the round. The Backend records the round and queues REVIEW unless AI mode is OFF or a permitted manual-review resubmission bypasses it.
4. Worker obtains a PostgreSQL lease, calls Agent, executes only authorized search requests, and validates the final response. It records findings and allowed item decisions in a transaction.
5. A clear missing item may return the request for targeted correction. A fully passing set remains IN_REVIEW until an accountant confirms the whole request. The client can submit later rounds against unresolved requirements.

4.2 Financial Evidence Rules

Monetary values travel as decimal strings. The Agent proposes SUM, SUBTRACT, or MULTIPLY relations with source-document operands; the Backend independently recalculates them using `Decimal` and checks evidence ownership. A reimbursement claim is not its own receipt: every claim line must match a distinct original receipt by amount and currency before automatic satisfaction. A wrong-period or wrong-entity file must not satisfy a requirement. Searches are bounded to current or historical records of the same authorized client; broadening a search does not broaden tenant access.

5 Product Capabilities

Folio turns a monthly request for accounting evidence into a controlled, traceable exchange between a firm and its client. The unit of work is a *collection request* for one client and one accounting month. Each request contains typed document requirements; each client submission forms a separate round. This structure lets staff identify what is still missing without losing earlier documents or decisions.

5.1 Role-Aligned Workspaces

Role	Available work
Firm administrator	Manage staff, clients, accountant assignments and invitations; work on all firm requests; withdraw a previous whole-request confirmation.
Accountant	Manage assigned clients' requests, review documents and evidence, request corrections, waive an item with a reason, and confirm a complete request.
Client administrator	See requests belonging to the client, submit documents, and manage that client's contacts and invitations.
Client submitter	View the client's requests, upload and submit material, and respond to a correction request.

The server rechecks firm, client, role, and assignment on every call; hiding a navigation item is only a usability measure. A client sees a filtered public timeline and client-facing messages, while internal notes,

model output, and accountant search history remain in the staff workspace.

5.2 Functional Modules

Module	Current user-visible behavior
Identity and onboarding	Sign in, recover a session through the refresh cookie, change a password, accept a one-time invitation, and request a password reset. Firm staff and client administrators create invitation links for their permitted audiences. Links are copied and shared manually; the application does not send invitation email.
Firm and client records	Firm administrators create clients, manage employee access, and assign accountants. The client detail view includes contacts and bank-account records. Client administrators can manage their own contacts; a submitter cannot administer other users.
Request preparation	Staff create a month-specific request with a due date, owner, client instructions, and required or optional typed evidence items. They can revise a draft, publish it, copy a prior request into a new draft, or cancel a request with a recorded reason. A published request becomes visible in the client portal.
Work queues and tracking	The staff home separates requests awaiting review, waiting for the client, due soon, and overdue. Collections can be searched, filtered by client/period/status/assignee/due date, sorted, and opened for an activity history. The portal shows the client only its own periods and checklist progress.
Evidence intake	The portal accepts PDF, PNG, and JPEG files by selecting or dragging them. A client can attach files directly to one requirement or use Smart upload to batch-stage files, inspect AI classification, move files between categories, and confirm placement. Upload progress, safety-scanning status, preview, download, and removal from an editable round are visible.
Assisted review	After submission, the Agent can read documents through OCR and return extracted fields, findings, cited files, and arithmetic relations. Backend search and validation constrain what the Agent can use; an accountant sees the explanation and source evidence before deciding. A failed or uncertain AI run leaves a manual review path.
Human resolution and history	Staff record an item as satisfied, needing action, or waived; return a request for another round; and confirm the whole request after all items are complete. AI may make a guarded item-level decision but cannot confirm the request. Earlier rounds, decisions, and activity events remain available.
Notifications and preferences	In-app notifications identify relevant request events and link back to the task. Users can filter unread items, update their password, and choose language, colour theme, and display mode in Account. Appearance choices are saved in the current browser.

5.3 AI Modes and Their Boundaries

When creating a request, staff choose *Manual only* (OFF), *Suggestions only* (SUGGEST), or *Auto-return issues* (AUTO_REVIEW). Manual only creates no post-submission review run. Suggestions only records analysis for the accountant without automatic item decisions. In automatic mode, the Backend may apply a sufficiently supported item decision or return a specific issue to the client; a fully passing round still waits for accountant confirmation. The separate *Cautious*, *Standard*, and *Efficient* preferences guide whether a clear gap should be requested from the client or routed to a human. They do not relax ownership checks, arithmetic validation,

or the human final approval requirement.

The accountant-facing AI result, source inspection, and financial reconciliation are shown in Figures 7–11. Folio currently prepares evidence for bookkeeping; it does not post journal entries, file tax returns, transfer funds, or control bank accounts. Email and Feishu delivery remain disabled even though the in-app notification and outbox records exist. The production case spot check in Section 9.2 is a quality observation, not a certification of AI accounting accuracy.

6 System Usage Guide

This guide follows the current English interface. The screenshots were captured from the live client and accountant workspaces using synthetic demonstration accounts and requests. They illustrate navigation, AI findings, source inspection, and human decision controls, not independently verified accounting quality. The described controls were checked against the implemented frontend and Backend contract; no production request was changed while preparing this guide. Figure 6 is also discussed in the case analysis because its historical AI pass must not be mistaken for verified correctness.

6.1 Getting Started and Reading Status

Open the HTTPS site and sign in with an invited account. Staff enter the firm workspace; client members enter the client workspace. The left navigation shows the actions allowed for that role. Use *Account* to change a password, language, colour theme, or light/dark/system display mode. Session refresh is handled by the application; signing out revokes the refresh session. If a user has not yet joined, an administrator must create an invitation and share its one-time link manually.

The request label in a list is a workflow summary, not an AI accuracy score. The operational sequence is DRAFT → OPEN → IN_REVIEW → CHANGES_REQUESTED → IN_REVIEW → READY_FOR_BOOKKEEPING. The correction loop may repeat; CANCELLED ends a request. Within IN_REVIEW, the interface may show *AI review in progress*, *Awaiting human review*, or *Awaiting confirmation*. These are derived display states. *Awaiting confirmation* means AI checks passed for that round, but a human has not yet approved the whole request. Individual requirements separately display *Pending*, *Received*, *Needs action*, *Satisfied*, or *Waived*.

6.2 Firm Administrator: Set Up a Client

1. In *Clients*, create the client record with its legal name and other required business details. Open the client detail page to review its contacts and, where relevant, bank-account records. The firm administrator assigns the responsible accountant; an accountant works only within the resulting client assignment.
2. Use the client invitation panel to invite the first *Client administrator*. Copy the generated link while it is displayed and share it through an approved channel. The link is one-time and time-limited; the system does not email it. The invited person sets a password or confirms an existing one on the acceptance page, then signs in.
3. In *Team*, invite accountants or other firm staff, review account status, and disable access when needed. A client administrator can subsequently open *Contacts* to invite *Client submitters* for that client. Invitations can be revoked or reissued; reissuing invalidates an older unused link.

Do not create a second active request for the same client and month: the Backend enforces one non-cancelled request per client-period pair. Check the existing request or use *Copy to next month* for a new period instead.

6.3 Accountant: Create, Check, and Publish a Request

Open *Collections* and select *New request*. The wizard has *Request*, *Documents*, and *Review* steps. Complete the fields as follows:

Field	Entry and operational effect
Client	Choose the legal client whose portal members must provide the evidence. This selection also constrains the accountant and document scope.
Period	Choose the accounting month, such as <i>November 2026</i> . It identifies the request and cannot be casually changed after creation.
Due date	Choose the client's expected submission deadline. The work queue uses it for due-soon and overdue views.
Assignee	Choose the accountant responsible for monitoring and review.
Instructions	Write a short client-facing description of the period and any special evidence expectation, for example: <i>Provide November bank statements and original receipts for the expense claim</i> . This text appears in the portal.
AI assistance	Select <i>Manual only</i> , <i>Suggestions only</i> , or <i>Auto-return issues</i> . For automatic mode, choose a review preference; it affects routing, not proof requirements.
Documents	Add each evidence item with a concrete title, document type, and <i>Required</i> or <i>Optional</i> flag. <i>November OCBC statement</i> is more actionable than <i>Bank files</i> . Separate an expense claim from its original receipts so both can be checked.

On *Review*, confirm the client, period, due date, instructions, policy, and every requirement. Select *Save draft*; then open the saved request and select *Publish*. The publish confirmation is the point at which the client can see and respond to the request. While still a draft, staff can edit details or remove an accidental requirement. Publishing locks down that preparation phase and records the event in the activity history. If a similar request already exists, use *Copy to next month* from the previous request and inspect the new draft before publication; old period-specific instructions may no longer apply.

The *Collections* list can be filtered by client, period, status, assignee, and due-date range, then sorted by due date, period, or last update. The staff *Work queue* groups requests awaiting review, waiting for a client, due soon, and overdue. Open a row to view its summary and activity; *Open full page* gives the complete request view. A request can be cancelled with a reason if it should not proceed; cancellation is an auditable business action, not deletion.

Figure 4: Live accountant Work queue. The EastBridge request is waiting for human confirmation after AI review.

6.4 Client: Find the Request and Interpret the Checklist

Select *Documents* in the client navigation. Filter by business, period, or status and open the period requested by the accountant. The list shows due date, current status, last update, and progress against required items (Figure 5). On the request page, check the accounting month, accountant, due date, *Instructions*, and the *Document checklist*. Expand each requirement to see its title, expected type, existing files, processing status, and any client-facing correction message (Figure 6). The *Timeline* shows public milestones such as publication, submission, AI review completion, correction request, and accountant confirmation.

Figure 5: Live client Documents list: period, status, due date, and required-item progress.

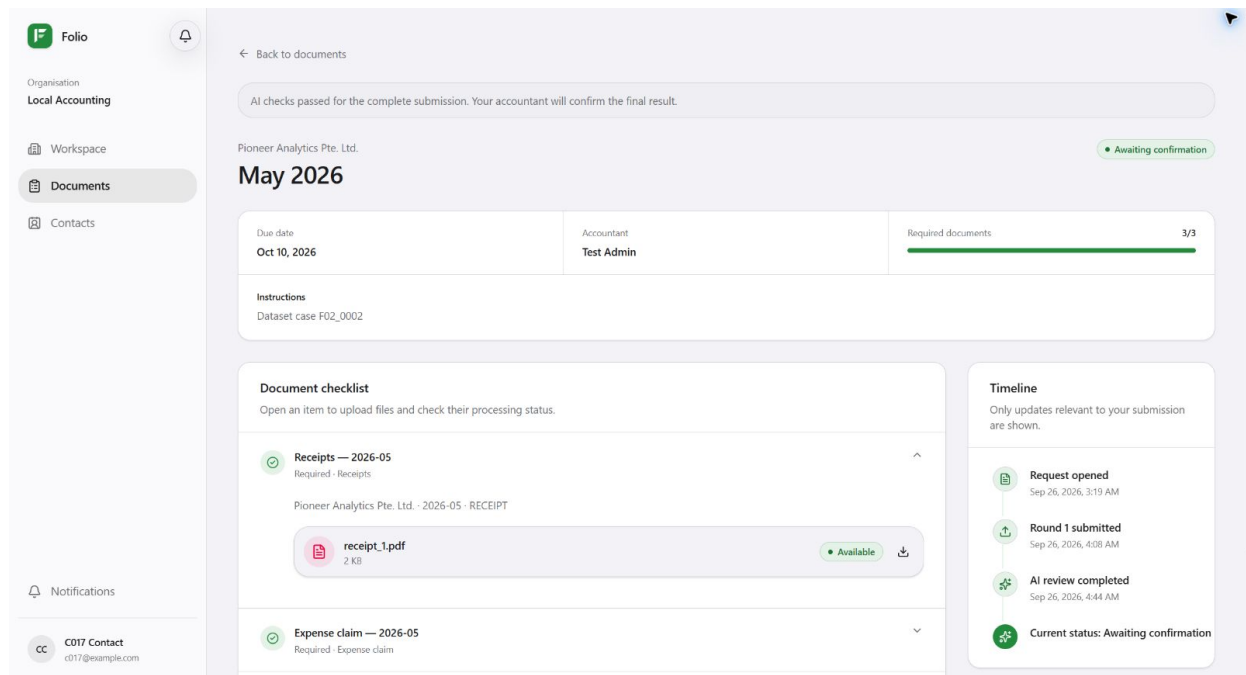


Figure 6: Live request page: instructions, expandable checklist, and public timeline. The displayed AI pass is not an independent accuracy assessment.

6.5 Client: Upload and Place Documents

There are two supported intake paths. For a known requirement, expand it and use *Choose or drop PDF and images*; select one or several PDF, PNG, or JPEG files. This directly associates each accepted file with that requirement. Use the filename and preview to verify that the correct original document was attached. A PDF of an expense claim does not replace the distinct receipts that support its line items.

For a mixed batch, select *Smart upload*. Choose or drop the files, review the selected-file list, and select *Confirm and analyze*. The system stages the uploads, scans them, and runs classification. On *Check AI classification*, inspect every suggested destination. Drag a misclassified file into the right category or select it and use *Move here*; keep invalid material out of the formal list. Select *Confirm and add to checklist* only after all categories are correct. Classification is a placement aid, not the later accounting review. If classification is disabled or fails, choose *Classify manually* and place the available files yourself. Unconfirmed staged files are not part of the formal requirement checklist.

Immediately after upload, a file can show *Scanning*; wait until it becomes *Available*. A *Failed* or unsafe file must be replaced with a valid original. While a round is editable, use preview or download to check the upload and *Remove from this submission* to exclude a mistaken attachment. The service checks extension, declared MIME, file signature, configured size, and malware before release to the review path. Reusing a previously available document may be indicated without another physical upload.

6.6 Client: Submit, Correct, or Challenge a Result

Before selecting *Submit for review*, check that every required item has an available file, that no upload is still scanning, and that each document sits under the right requirement. Add an optional *Note for your accountant* with relevant context for this submission round. Confirm the submission dialog. The round becomes read-only, enters *In review*, and is separately recorded in the timeline. Depending on the request policy, AI analysis may start asynchronously; a successful upload or submission response does not mean the accountant has accepted the evidence.

If the request changes to *Changes requested*, open each *Needs action* item and read the visible message. Add the missing or corrected file, then submit another round. Earlier rounds remain in the staff record. If the

automated return itself appears wrong and the portal offers *Request an accountant's review*, the client may keep the existing valid files, select that option, and submit unchanged. That exceptional round skips another AI review and goes to the accountant. The option appears only for an eligible AI-returned request, not for every correction. Do not repeatedly upload the same evidence merely because the interface says *Awaiting human review*; that status means a person must decide.

6.7 Accountant: Review Evidence and Record Decisions

From *Work queue* or *Collections*, open an *In review* request and select *Open review*. Choose the latest *Submission round*; earlier rounds are read-only context. The review screen has the requirement list, files and AI analysis, and decision controls. Read the client's round note, open the original document preview, and compare it with the stated requirement. The AI panel can show extraction fields, entity/period checks, evidence references, search findings, amount operands and differences, or a reason for escalation. Follow the cited file to the original document. A green AI label alone is not evidence that the result is correct; Section 9.2 gives a concrete historical counterexample.

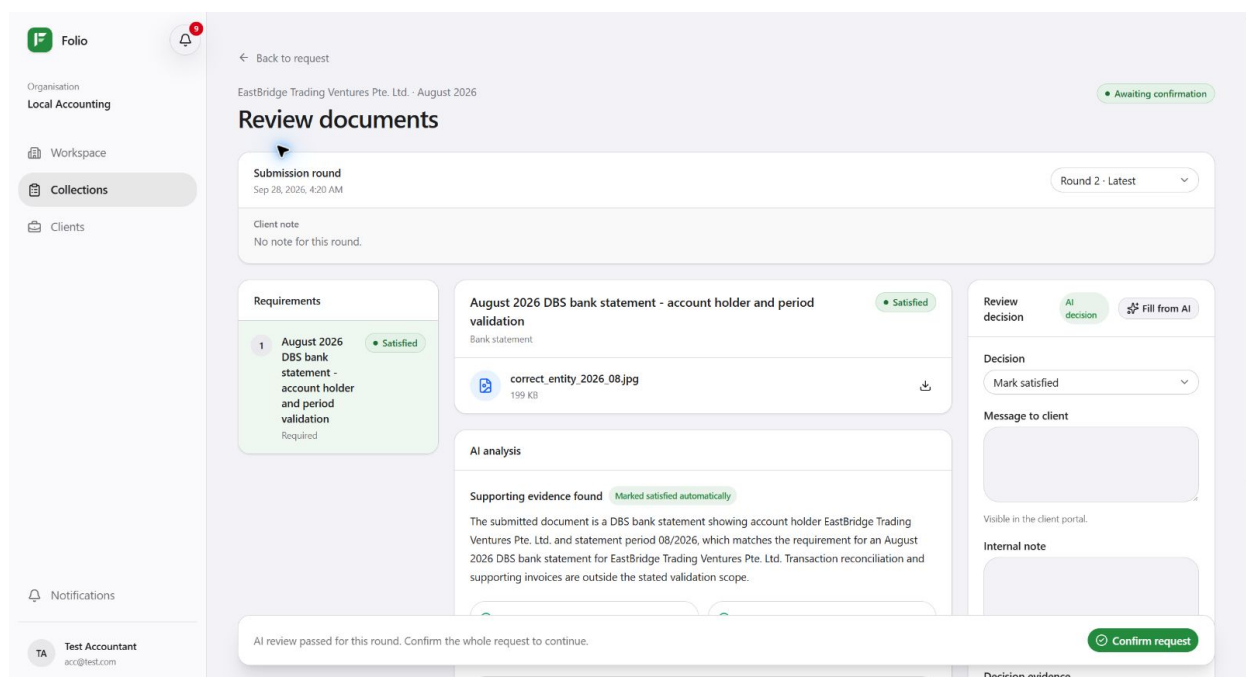


Figure 7: Live accountant review before whole-request confirmation: latest round, requirement, submitted file, AI conclusion, and human decision controls.

For each item, choose *Mark satisfied* only if its source evidence actually meets the requirement. Choose *Request action* for a specific remediable issue, select an issue category (for example missing, wrong period, wrong entity, unreadable, or incomplete), and write a precise *Message to client*. Choose *Waive requirement* only with a reason. *Internal note* remains staff-only. Select the files that support the decision under *Decision evidence*, then *Save decision*. *Fill from AI* copies a suggestion into the editable form; inspect and change it before saving. Manual decisions can supersede an AI item decision without removing its earlier audit record.

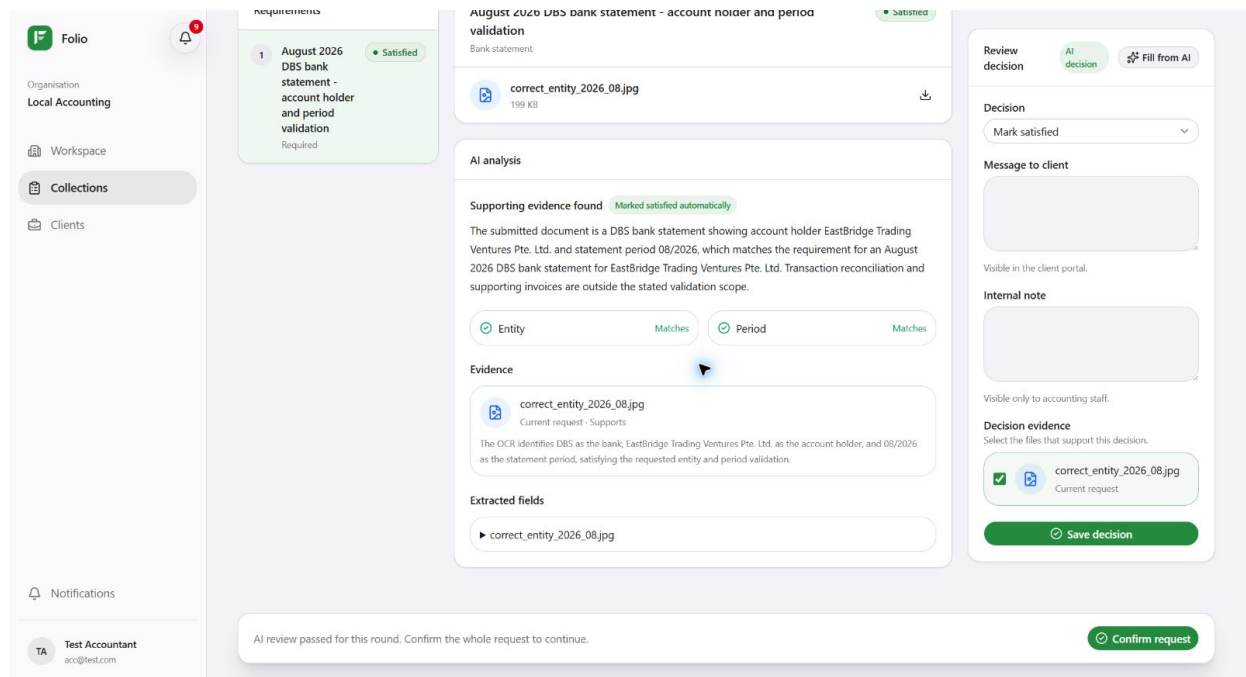


Figure 8: The same live review scrolled to entity/period checks, cited source, extracted fields, selected decision evidence, and Save decision.

Select a cited file to open the source-document preview before deciding. The preview in Figure 9 exposes the statement holder and month that the AI explanation claims to have found; the accountant must compare those visible facts, rather than approving from the summary alone.

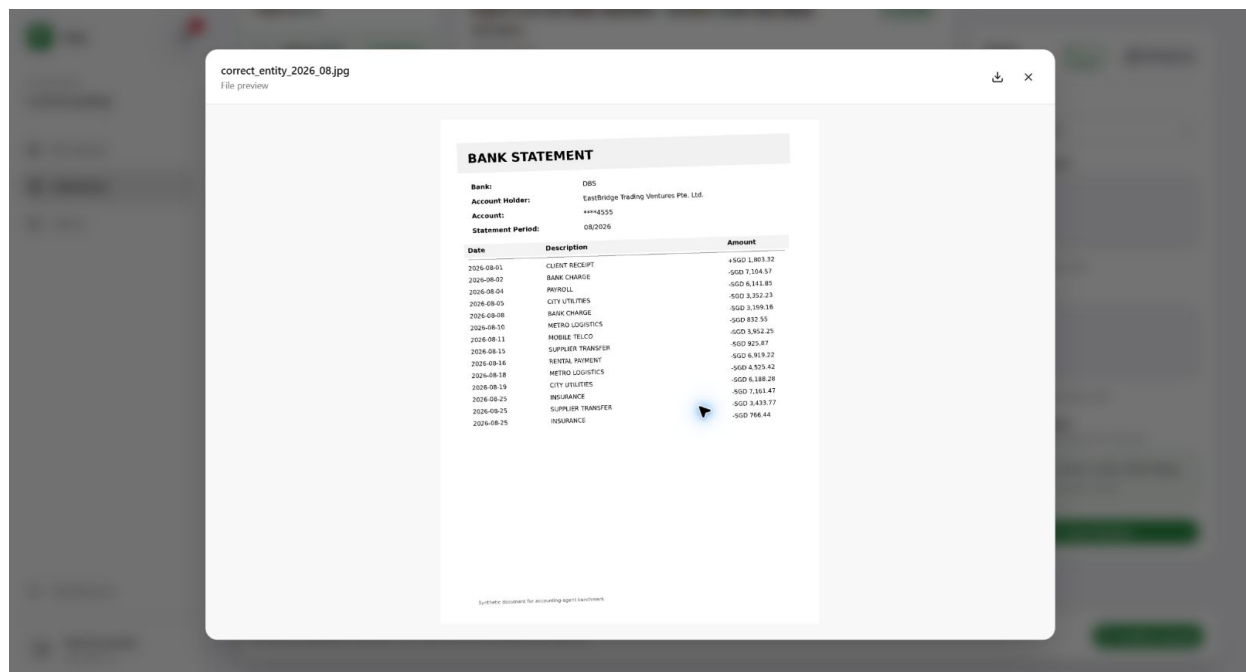


Figure 9: Accountant-side preview of the cited synthetic bank statement used for entity and period validation.

After each item has a saved decision, use *Return to client* when at least one item needs action, entering a clear overall reason. The client receives the item-specific message and can create the next round. When every listed item is *Satisfied* or *Waived*, choose *Confirm request*; the request reaches *Confirmed / Ready for bookkeeping*. This is a human whole-request action even if all item checks were automated. Only a firm administrator can later use *Withdraw confirmation*, with a reason, to return the request to review. If the AI

run fails or the evidence is ambiguous, proceed with the same manual item-review path rather than treating run failure as client error.

6.8 Worked AI Review: Reimbursement Evidence

The live reimbursement demonstration request is a more informative review example than a single-document pass. Its latest submitted round contains a bank statement, an employee expense claim, and original receipts. Figure 10 shows the accountant's item review and AI narrative. In Figure 11, the analysis cites each receipt separately and presents two Backend-checked relationships: $\text{SGD } 254.14 + 19.57 + 67.49 = 341.20$, then the SGD 341.20 claim against the SGD 341.20 bank payment with zero difference. The right-hand panel records the files selected as decision evidence. The request was subsequently confirmed by an accountant. This screenshot is an operational illustration; its figures and citations should still be checked against the original files, and it is excluded from the diagnostic answer-quality rate in Section 9.

The screenshot displays the Folio AI review interface for a reimbursement request. The interface is divided into a sidebar and a main content area. The sidebar on the left contains navigation options: Folio, Organisation (Local Accounting), Workspace, Collections, Clients, Notifications, and a Test Accountant (acc@test.com). The main content area is titled 'Review documents' and shows a 'Submission round' for 'Sep 28, 2026, 6:49 AM'. It lists requirements: 'November 2026 bank statement - employee reimbursement reconciliation' (Required, Satisfied), 'Employee expense claim - AISHA RAHMAN' (Optional, Satisfied), and 'Original receipts supporting the employee expense claim' (Optional, Satisfied). The 'AI analysis' section shows 'Supporting evidence found' and 'Marked satisfied automatically'. It details the bank statement for UOB account holder Keystone Projects Solutions Pte. Ltd., period 11/2026, including the 2026-11-28 payment to AISHA RAHMAN for SGD 341.20. The AI analysis also shows 'Entity' and 'Period' checks, both marked 'Matches'. The 'Review decision' section shows 'AI decision' and 'Mark satisfied'. It includes fields for 'Message to client', 'Internal note', and 'Decision evidence'.

Figure 10: Live multi-document AI review: bank, claim, receipts, entity/period checks, and the recorded AI item decision. The confirmed round is read-only.

The screenshot displays the Folio application interface for reviewing a reimbursement request. The left sidebar contains navigation links: Folio, Organisation, Local Accounting, Workspace, Collections (highlighted), and Clients. The main content area shows a list of receipts and amounts. The right sidebar includes an internal note, a decision evidence list, and a save decision button.

Receipts:

- expense_claim.pdf** (Current request - Supports): Expense claim EC-239 is for Aisha Rahman, total SGD 341.20, with three expense lines.
- receipt_1.jpg** (Current request - Supports): OfficeHub receipt dated 2026-11-13 for SGD 254.14 matches claim line 1 by amount and currency.
- receipt_2.jpg** (Current request - Supports): OfficeHub receipt dated 2026-11-19 for SGD 19.57 matches claim line 2 by amount and currency.
- receipt_3.jpg** (Current request - Supports): OfficeHub receipt dated 2026-11-10 for SGD 67.49 matches claim line 3 by amount and currency.

Amounts:

- $254.14 + 19.57 + 67.49 = 341.20$ SGD
Expected: 341.20 - Difference: 0.00
- $341.20 - 341.20 = 0.00$ SGD
Expected: 0.00 - Difference: 0.00

Arithmetic verified by the backend. Verify the source figures against the documents.

Extracted fields:

- bank_2026_11.jpg

Decision evidence: Select the files that support this decision.

- ☒ bank_2026_11.jpg (Current request)
- ☒ receipt_1.jpg (Current request)
- ☒ receipt_3.jpg (Current request)
- ☒ receipt_2.jpg (Current request)
- ☒ expense_claim.pdf (Current request)

Internal note: Visible only to accounting staff.

Save decision

Figure 11: Evidence and monetary checks for the reimbursement: three distinct receipts, two arithmetic relations, and the accountant-visible decision-evidence list.

6.9 Contacts, Notifications, and Personal Settings

A *Client administrator* opens *Contacts* to review active contacts and create a client-submitter invitation. Select the role, enter the intended email, create the invitation, and copy the one-time link immediately; no email is sent automatically. *Notifications* provides *All* and *Unread* views for request events and links back to the relevant task. In *Account*, users can change a password and select English or Simplified Chinese, a colour theme, and a display mode. Appearance and language preferences are stored for the current browser.

6.10 Common Blocking Conditions

Symptom	What the user should check
The client cannot find a request	Staff should check that the request was <i>Published</i> , belongs to the correct client and month, and has not been cancelled. A saved draft is staff-only.
Submit is blocked	Ensure every required item has an available file, not one still scanning or failed. Confirm Smart upload placement before submission.
AI classification is unavailable	Use manual placement, then confirm the categories. A classification failure does not require a new business request.
AI review failed or remains uncertain	The accountant can inspect the originals and record manual decisions. The client should wait for an explicit correction request.
An AI return appears wrong	On an eligible AI-returned request, use <i>Request an accountant's review</i> and resubmit valid evidence without forcing a duplicate upload.
A staff save reports a conflict	Another action changed the request version. Reload the latest state, review the differences, then save again.
Confirmation is disabled	Finish each listed item's review; all listed items must be satisfied or waived, and the request must be in review.

7 API Documentation

7.1 Public Backend Contract

The business API is versioned under `/api/v1`. JSON property names use `snake_case`; the frontend converts naming at its Axios boundary. Access tokens identify users, while refresh state is revocable. Every operation is authorized by role, firm, and client assignment. Collection creation returns HTTP 201; asynchronous upload returns HTTP 202. The Backend includes X-Request-ID on responses and uses a stable error envelope. Lists are paginated and filterable. State-changing actions carry optimistic version; stale versions return HTTP 409. Important writes use Idempotency-Key so retries cannot duplicate decisions.

```
{ "code": "VERSION_CONFLICT", "message": "Requirement has changed",
  "details": { ... }, "request_id": "<request-id>" }
```

7.2 Authentication and Error Handling

POST `/auth/login` accepts email and password. Its JSON response contains a short-lived bearer `access_token`, `expires_in`, and the user's firm and client memberships. Send the access token in `Authorization: Bearer <token>` for protected calls. The rotating refresh token is an HTTP-only, SameSite Strict cookie scoped to `/api/v1/auth`; POST `/auth/refresh` and `/auth/logout` validate the browser origin. Never expose the refresh token to JavaScript or persist the access token in a shared server log.

Typical response classes are 401 for missing or expired authentication, 403 for an authenticated user outside the permitted firm/client/role, 404 for an inaccessible resource, 409 for a stale version or invalid state transition, 415 for a mismatched upload type, 422 for missing required evidence or invalid input, and 429 for a rate limit. Clients should key recovery logic on the stable code, display the safe message, and retain `request_id` for support. A 202 upload response means the file is quarantined and still awaits scanning; wait for AVAILABLE before submitting.

7.3 Primary Resource Operations

The table gives the main integration path; Appendix B lists all 68 Backend routes extracted from the checked OpenAPI schema.

Method	Path (under <code>/api/v1</code>)	Purpose
POST	<code>/auth/login, /auth/refresh, /auth/logout</code>	Session lifecycle
GET	<code>/me, /clients, /users</code>	Current identity and staff records
POST	<code>/clients, /clients/{client_id}/invitations</code>	Client administration
GET/POST	<code>/collection-requests</code>	List or create requests
GET/PATCH	<code>/collection-requests/{request_id}</code>	Read or edit a draft
POST	<code>/collection-requests/{request_id}/publish</code>	Make request client-visible
POST	<code>/collection-requests/{request_id}/requirements</code>	Add evidence item
GET	<code>/portal/collection-requests/{request_id}</code>	Client request view
POST	<code>/portal/collection-requests/{request_id}/documents</code>	Multipart upload
POST	<code>/portal/collection-requests/{request_id}/classification-runs</code>	Begin file placement session
POST	<code>/portal/collection-requests/{request_id}/classification-runs/{run_id}/confirm</code>	Confirm placement
POST	<code>/portal/collection-requests/{request_id}/submit</code>	Submit a new round
GET	<code>/collection-requests/{request_id}/review-runs</code>	AI run history
GET	<code>/collection-requests/{request_id}/review</code>	Staff review view

Method	Path	Purpose
POST	/requirements/{requirement_id}/review	Human item decision
POST	/collection-requests/{request_id}/approve	Human whole-request confirmation
GET	/notifications	In-app notification list
GET	/health/live, /health/ready	Liveness and dependencies

7.4 Representative Request Contracts

Creation accepts `client_id`, a month-start `period`, timezone-aware `due_at`, `scope_note`, optional `assignee_id`, AI mode and preference, and one to 100 typed requirements. The Backend derives a bank reconciliation analysis type for bank statements unless explicitly specified. The following is schematic; IDs and dates must be replaced with authorized values.

```
POST /api/v1/collection-requests
Content-Type: application/json
Idempotency-Key: <unique-request-key>

{
  "client_id": "<uuid>", "period": "2026-11-01",
  "due_at": "2026-12-05T17:00:00+08:00",
  "scope_note": "Provide November bank and expense evidence.",
  "ai_mode": "AUTO_REVIEW", "review_preference": "STANDARD",
  "requirements": [
    {"type": "BANK_STATEMENT", "title": "November bank statement", "required": true},
    {"type": "EXPENSE_CLAIM", "title": "Employee expense claim", "required": true}
  ]
}
```

The client upload endpoint uses `multipart/form-data` with a file and either a target `requirement_id` or a `classification_run_id` for staging. A classification run is created, uploaded to, started, polled, then confirmed or manually placed. Formal submission uses `POST /portal/collection-requests/<request_id>/submit` with an optional note and `manual_review_requested`. Staff item review requires the requirement version, `submission_id`, decision, optional issue code/message, and explicit evidence document IDs. Whole-request approval requires the request version; incomplete items return HTTP 422.

The creation response is a request detail with an `id`, `version`, `DRAFT` status, and typed requirements with their own IDs and versions. Publish through the action endpoint before a client can use it. Portal submission returns the updated request; review processing continues asynchronously, so poll the request or review-run resource instead of treating the submit response as an AI verdict. Staff approval returns `READY_FOR_BOOKKEEPING` only after every requirement has reached `SATISFIED` or `WAIVED`. The machine-readable `/openapi.json` remains authoritative for all field-level schemas.

7.5 Private Backend-Agent Contract

`POST /v1/analyze` is private to the internal network. Its `Idempotency-Key` has the form `run_id:turn`. `CLASSIFY` sends document references and requirement choices; its response contains `classifications`. `REVIEW` sends authorized context, document references, requirement targets, and at most three recorded search turns; its response contains `extractions`, `findings`, and optionally one search action. Files are identified by storage key and hash, then independently checked by the Agent. `POST /v1/analyze-inline` is a separately authenticated Base64 interface for trusted integrations and is not exposed by the current Compose deployment. Agent liveness and readiness are `/health/live` and `/health/ready`; Backend readiness does not depend on Agent availability.

8 Folio-Reasoner-8B and Model Evaluation

8.1 Model Role and Provider Boundary

The AI layer is defined by the task contract rather than by a single language-model vendor. For CLASSIFY, it must place each file under a proposed requirement; for REVIEW, it must return structured findings, cited document IDs, monetary relations, or a bounded search request. The Backend retains document authorization, search execution, arithmetic validation, workflow transitions, and final accountant approval. Folio uses Folio-Reasoner-8B as its selected accounting specialist. The Agent has a hosted-provider path, a REMOTE path, and a development-only mock; each must satisfy the same output schema. Connecting another model requires a provider adapter or a service implementing the REMOTE envelope. Changing a model name alone does not make arbitrary vendor APIs compatible.

Folio-Reasoner-8B is the team’s Qwen3-8B-based specialist for this accounting task contract. The model study below reports its training and evaluation in the team’s executable case environment. The dated production observations in Section 9 concern a separately configured demonstration Agent and are not measurements of Folio-Reasoner-8B. The platform can keep OCR and the reasoning model as separate services, provided the selected model receives controlled evidence and returns the validated task schema.

8.2 Accounting Dataset and Executable Environment

The *Accounting Agent 2,000-Case Dataset v5* contains 2,000 synthetic business cases across 100 companies and 12 scenario families. Its 12,531 evidence files comprise 8,414 PDFs and 4,117 JPGs. There are 1,217 atomic and 683 composite cases; 100 cases contain concurrent errors. Language labels cover 600 English, 600 Chinese, 600 Malay, and 200 mixed-language cases. Difficulty labels cover 600 simple, 1,000 medium, and 400 hard cases; a case has between one and 16 documents. The corpus includes 5,833 distractor documents, so finding the right evidence is part of the task.

The scenario families form three layers: document validation (missing evidence, wrong period, wrong entity); core reconciliation (many invoices to one payment, instalments, historical invoices, and credit-note adjustments); and specialized accounting (platform settlements, employee expenses, loan repayments, foreign exchange, and progress billing with retention). The data combines entity and period checks with document provenance and financial relationships. A case contains a client profile, a task, evidence, reference outcomes, and an interaction schedule, not merely a static question and answer.

Evidence zone	Documents	Availability in the case environment
Initial	2,040	Visible at the start.
Searchable current	8,127	Retrieved from current-period records.
Searchable history	234	Retrieved for cross-period reasoning.
Client-held	2,130	Released after a targeted follow-up.

Table 4: Visibility schedule in the synthetic agent environment. The 1,400 state-changing cases require follow-up or new input; 600 can be resolved from existing evidence.

Company IDs are disjoint across the original development (1,400 cases, 60 companies), validation (300, 12 companies), and test (300, 28 companies) partitions. The fixed FolioBench-120 evaluation suite samples ten cases from each of the 12 families within the test partition, spanning 26 companies and all four language labels. Sampling was fixed by seed and difficulty allocation before model scores were compared. This is a synthetic, scenario-balanced case-level benchmark; it does not measure performance on real client records.

8.3 Training Procedure

The specialist starts from Qwen3 8B and is trained in two stages. For supervised fine-tuning, DeepSeek V4.0 Pro generated eight candidate trajectories for each of 240 development cases. The team retained successful, manually screened trajectories and trained on 128 demonstrations for one epoch, with batch size eight and learning rate 10^{-5} on one H100 GPU. The resulting checkpoint reached 24.6% validation accuracy in the team’s records.

The second stage continues from that checkpoint on the remaining 1,160 development cases using DAPO (Yu et al., 2025, arXiv:2503.14476). Each of 88 policy updates draws 16 tasks with eight rollouts per task, or 128 trajectories. Reward is sparse: one for a correct task outcome and zero otherwise, with no intermediate step reward. The reported learning rate is 10^{-6} ; training used eight H100 GPUs on one node with Fully Sharded Data Parallel. Validation accuracy rose to 66.4% at update 88, a 41.8-percentage-point gain over the post-SFT checkpoint. These are the team’s reported settings and learning-curve values, not a run reproduced for this document.

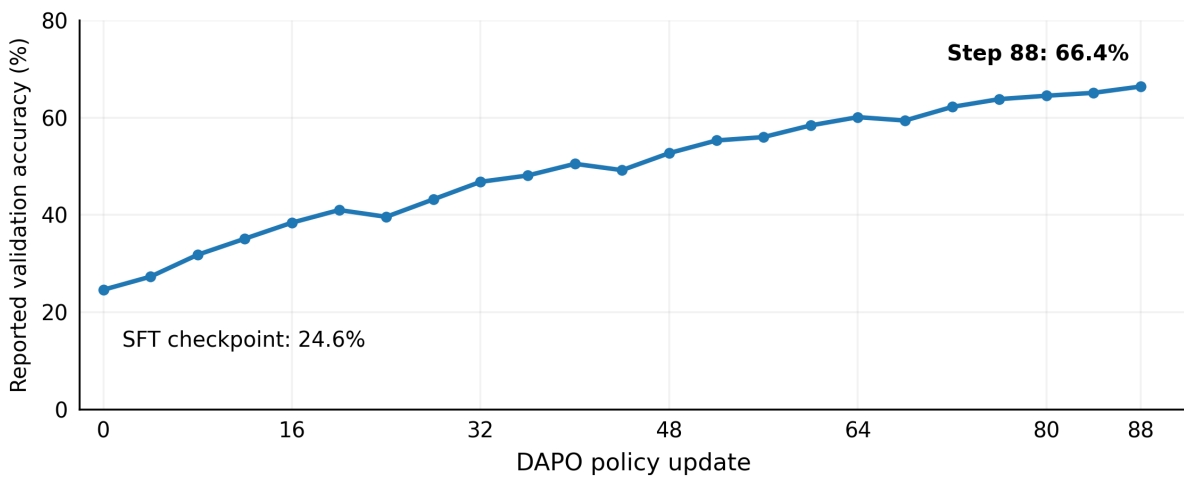


Figure 12: Team-reported validation accuracy through 88 DAPO updates. The starting point is the SFT checkpoint; intermediate points are measured every four updates.

8.4 FolioBench-120 Results and Interpretation

On the fixed 120-case synthetic suite, the team reports 79 correct cases for Folio-Reasoner-8B (65.8%) and 68 for its demonstration-generating teacher, DeepSeek V4.0 Pro (56.7%), a difference of 11 cases or 9.2 percentage points before display rounding. The specialist leads in 11 scenario categories and ties one. Table 5 gives the reported per-category comparison; each category has ten cases, so its entries are percentages and overall accuracy is also the equal-category macro-average.

Scenario	Folio 8B	DeepSeek	27B	14B	8B
G01 Missing evidence	70	60	40	0	0
G02 Wrong period	100	90	80	80	0
G03 Wrong entity	100	90	60	40	0
B01 Many invoices, one payment	100	90	50	0	0
B02 One invoice, many payments	60	50	20	0	0
B03 Historical invoice	20	20	0	0	0
B04 Credit note	40	30	10	0	0
F01 Platform settlement	50	40	20	0	0
F02 Employee reimbursement	30	20	10	0	0
F03 Loan repayment	80	70	30	0	0
F04 Foreign exchange	90	80	30	0	0
F05 Progress billing	50	40	10	0	0
Overall (120 cases)	65.8	56.7	30.0	10.0	0.0

Table 5: Team-reported FolioBench-120 accuracy in percent. *DeepSeek* is DeepSeek V4.0 Pro; the other baselines are Qwen3 27B FP8, Qwen3 14B, and Qwen3 8B. Ten cases contribute to each row.

The comparison uses a common task harness, document/OCR content, tools, and outcome scorer with each model’s native or recommended template. The 65.8% figure is an evaluation result for this trained checkpoint under that harness, not the live system’s success rate. The operational completion and diagnostic answer-quality rates in Section 9 use different configurations, evidence schedules, and assessment criteria. Before treating this model as a release-quality replacement, the same pinned model and Agent adapter must be tested end to end against held-out trajectories, exact evidence IDs, latency, retries, and accountant review outcomes. Results in this section are transcribed from the team’s *Folio Reasoner 8B Technical Brief Refined*; raw checkpoints and score records were not independently reproduced here.

9 Technical Report: Operational Results

9.1 Completion and Answer-Quality Rates

The demonstration Agent achieved an *execution completion rate* of **86.4%** for REVIEW and **72.7%** for CLASSIFY, excluding cancelled classification work. These rates describe whether an analysis completed, not whether its accounting conclusion was correct. They reflect historical configurations and retries and must not be attributed to the Folio-Reasoner-8B checkpoint evaluated in Section 8.

A separate comparison with reference outcomes found a **33.3% strict final-answer success rate**. A result counted as successful only when its resolution direction, cited source evidence, and monetary relationship agreed with the reference answer, without using distractors or treating a claim as its own support. This diagnostic rate comes from a small, non-representative sample. The demonstration workflow did not reproduce every reference evidence-visibility turn, so the rate is neither held-out model accuracy nor full-trajectory success.

9.2 Illustrative Review Findings

Scenario	Finding compared with reference evidence	Assessment
Progress billing and retention	A project certificate's gross amount less retention matched the bank payment. The final finding cited the claim, certificate, and bank evidence, although an earlier follow-up was unnecessary.	Correct final relationship; inefficient path.
Invoice paid in instalments	The payments reconciled to the invoice, but the final finding also cited unrelated files excluded by the reference answer.	Correct arithmetic; incorrect evidence selection.
Employee reimbursement	The claim total reconciled arithmetically, but the receipt operands pointed to the claim rather than independent original receipts.	Incorrect receipt provenance despite a passing item status.

A live reimbursement review provides a separate operational illustration. Its latest analysis cites distinct original receipts, reconciles them to the claim and bank payment with zero difference, and records the accountant's confirmation. Figures 10 and 11 show the staff-side review and monetary checks. This illustrated request has no corresponding reference trajectory and is excluded from the answer-quality rate. Conversely, Figure 6 shows a historical request displaying a pass despite incomplete cited receipt evidence; a green UI status is not a substitute for answer validation.

9.3 Interpretation and Next Verification Gate

The observed errors show why a SUCCEEDED analysis and a green item status cannot be used as accuracy metrics. Receipt provenance and distractor exclusion must be tested independently of arithmetic. A release-quality evaluation should replay held-out cases on a pinned Backend, Agent, and Folio-Reasoner-8B configuration, preserve the evidence-visibility schedule, compare final actions and exact evidence IDs with reference answers, and report retries, timeouts, scenario breakdowns, and language breakdowns. FolioBench-120 is a separate synthetic model evaluation, not an end-to-end operational success rate. Selected development examples do not establish stability across the full case library.

10 Deployment and Operations

10.1 Build and Release Topology

The four application images are frontend, backend, Agent, and Nginx. Backend, Worker, and one-shot migration use the same backend image. PostgreSQL, Redis, and ClamAV use versioned official images. GitHub Actions tests and builds, assumes an AWS role via OIDC, and pushes immutable Git-SHA tags to ECR; it does not SSH into production. Verify the deployed image digests before comparing operational results with source code, and apply the current database migration as part of release.

10.2 Production Configuration

On the server, keep `/opt/acc-system/.env.prod` outside source control. Set database secrets (`DATABASE_URL`, `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`) and cache secrets (`REDIS_URL`, `REDIS_PASSWORD`). Set a random `JWT_SECRET`, `FRONTEND_ORIGIN` to the exact HTTPS origin, and `COOKIE_SECURE=true`. Pin immutable `BACKEND_IMAGE`, `FRONTEND_IMAGE`, `AGENT_IMAGE`, and `NGINX_IMAGE` tags.

For the selected Folio-Reasoner-8B deployment, place an authenticated serving adapter behind the Agent's REMOTE provider. The adapter must accept the Agent's complete-file CLASSIFY/REVIEW JSON envelope,

perform or obtain OCR as required, and return the exact validated response schema; it is not a direct arbitrary chat-completions URL. Configure `AGENT_REVIEW_PROVIDER=REMOTE`, `MODEL_API_URL`, `MODEL_HEALTH_URL`, and `MODEL_API_KEY` only after the adapter is deployed. The readiness endpoint must return authenticated 200 with `"status":"ok"`. Classification may use a separate provider. Merely setting `MODEL_NAME` does not switch the current Agent path to Folio-Reasoner-8B. The historical demonstration rates in Section 9 are not a deployment check for the specialist model.

Production must run ClamAV and set `CLAMAV_HOST=clamav`; the Worker refuses to release unscanned files. Configure the domain-specific Nginx TLS files and certificate mounts through `compose.tls.yml`. Cloudflare must validate the origin certificate in Full (strict) mode. Do not expose Backend, Agent, PostgreSQL, or Redis container ports on the host.

For a fresh installation, create the four ECR repositories in `ap-southeast-1`, configure GitHub Actions OIDC and the repository variables specified in the Backend deployment guide, and build/push immutable Git-SHA images. Supply the server's Docker daemon a short-lived ECR login token from an AWS SSO-authenticated workstation; do not place permanent AWS keys in `.env.prod`. Issue the first origin certificate using Certbot's standalone challenge, copy `nginx/tls/https.conf.example` to `nginx/tls/runtime/https.conf`, set the real hostname, and mount certificates with the TLS overlay. After migration, run the one-time bootstrap tool with `BOOTSTRAP_FIRM_NAME`, `BOOTSTRAP_ADMIN_NAME/EMAIL/PASSWORD` set to intended initial values. The deployed renewal job uses webroot and reloads Nginx; check its scheduled log and certificate expiry.

10.3 Controlled Deployment Procedure

From `/opt/acc-system`, after refreshing the short-lived ECR login token, pin the intended image SHAs in `.env.prod`. Apply the migration once before refreshing services. Keep the TLS overlay and malware-scan profile on all relevant Compose commands.

```
docker compose --profile malware-scan --env-file .env.prod \
  -f compose.yml -f compose.tls.yml pull
docker compose --profile malware-scan --env-file .env.prod \
  -f compose.yml -f compose.tls.yml run --rm migrate
docker compose --profile malware-scan --env-file .env.prod \
  -f compose.yml -f compose.tls.yml up -d --remove-orphans
docker compose --profile malware-scan --env-file .env.prod \
  -f compose.yml -f compose.tls.yml ps
```

Check the public HTTPS page, Backend `/api/v1/health/live` and `/api/v1/health/ready`, Agent internal `/health/ready`, PostgreSQL and Redis health, and a staff/portal smoke workflow. Backend readiness confirms its own database and Redis dependencies; the Agent may fail independently while manual operations remain available. Watch Worker run leases, Agent errors, upload scans, disk capacity, and certificate renewal output.

10.4 Rollback, Backup, and Known Gaps

Application rollback restores the previous immutable image tags and runs Compose again. A database migration cannot be reversed merely by restoring old images; take a PostgreSQL backup before a destructive migration and prefer backward-compatible schema changes. The documents volume requires encrypted snapshots and off-host backup; restore drills, central metrics and alerts, and formal load testing remain work to complete. In-app notifications are delivered; Email/Feishu outbox delivery is suppressed. The deployed TLS renewal job exists, but its alerting and failure exercise still require verification. Do not present the current deployment as a fully production-hardened accounting system without those controls and an independent security review.

A Source Register and Measurement Notes

The system design specification and Folio architecture drawings provide the topology; the Backend, Agent, and Frontend sources provide the interface and deployment details. The model-training settings and FolioBench-120 scores in Section 8 come from the team-supplied `Folio_Reasoner_8B_Technical_Brief_Refined.docx`; this report did not reproduce its checkpoints or scoring run. Reference answers for the diagnostic review came from the local v5 accounting case dataset. The operational rates summarize historical demonstration activity. Answer-quality assessment compares the final action, source evidence, and monetary relationship with each reference outcome; the underlying records and credentials are not included here.

B Complete Backend Endpoint Inventory

This inventory was generated from the Backend OpenAPI schema at the stated code revision. It lists method and route; the code-generated `/openapi.json` remains the machine-readable field reference.

Method	Route after <code>/api/v1</code>	Operation
POST	<code>/auth/login</code>	Login
POST	<code>/auth/refresh</code>	Refresh
POST	<code>/auth/logout</code>	Logout

Method	Route	Operation
GET	/me	Me
PATCH	/me/password	Change Password
POST	/auth/invitations/accept	Accept Invitation
POST	/auth/password-reset/request	Request Password Reset
POST	/auth/password-reset/confirm	Confirm Password Reset
GET	/users	List Users
POST	/users/invitations	Invite Staff
GET	/users/invitations	List Staff Invitations
PATCH	/users/<user_id>	Update User
GET	/clients	List Clients
POST	/clients	Create Client
GET	/clients/<client_id>	Get Client
PATCH	/clients/<client_id>	Update Client
GET	/clients/<client_id>/members	List Client Members
PATCH	/clients/<client_id>/members/<user_id>	Update Client Member
GET	/clients/<client_id>/assignments	Get Assignments
PUT	/clients/<client_id>/assignments	Replace Assignments
POST	/clients/<client_id>/invitations	Invite Client Member
GET	/clients/<client_id>/invitations	List Client Invitations
POST	/users/invitations/<invite_id>/revoke	Revoke Staff Invitation
POST	/users/invitations/<invite_id>/resend	Resend Staff Invitation
POST	/clients/<client_id>/invitations/<invite_id>/revoke	Revoke Client Invitation
POST	/clients/<client_id>/invitations/<invite_id>/resend	Resend Client Invitation
GET	/clients/<client_id>/bank-accounts	List Bank Accounts
POST	/clients/<client_id>/bank-accounts	Create Bank Account
PATCH	/clients/<client_id>/bank-accounts/<bank_id>	Update Bank Account
GET	/collection-requests/dashboard	Dashboard
GET	/collection-requests	List Collections
POST	/collection-requests	Create Collection
GET	/collection-requests/<request_id>	Get Collection
PATCH	/collection-requests/<request_id>	Update Collection
POST	/collection-requests/<request_id>/publish	Publish Collection
POST	/collection-requests/<request_id>/cancel	Cancel Collection
POST	/collection-requests/<request_id>/copy	Copy Collection
POST	/collection-requests/<request_id>/requirements	Add Requirement
GET	/collection-requests/<request_id>/events	List Events
PATCH	/requirements/<requirement_id>	Update Requirement
DELETE	/requirements/<requirement_id>	Delete Requirement
GET	/portal/collection-requests	List Collections
GET	/portal/collection-requests/<request_id>	Get Collection
POST	/portal/collection-requests/<request_id>/documents	Upload Document
GET	/portal/documents/<document_id>	Get Document
GET	/portal/document-links/<link_id>	Get Document Link
DELETE	/portal/document-links/<link_id>	Exclude Document
GET	/portal/document-links/<link_id>/download	Download Document
POST	/portal/collection-requests/<request_id>/submit	Submit Collection
GET	/collection-requests/<request_id>/review-runs	Get Review Runs
POST	/collection-requests/<request_id>/review-runs/<run_id>/retry	Retry Review
GET	/collection-requests/<request_id>/review	Get Review
POST	/requirements/<requirement_id>/review	Review Requirement
POST	/collection-requests/<request_id>/request-changes	Request Changes
POST	/collection-requests/<request_id>/approve	Approve
POST	/collection-requests/<request_id>/reopen	Reopen

Method	Route	Operation
GET	/documents/<document_id>/download	Download Document
POST	/portal/collection-requests/<request_id>/classification-runs	Create Run
GET	/portal/collection-requests/<request_id>/classification-runs/<run_id>	Get Run
POST	/portal/collection-requests/<request_id>/classification-runs/<run_id>/start	Start Run
POST	/portal/collection-requests/<request_id>/classification-runs/<run_id>/cancel	Cancel Run
POST	/portal/collection-requests/<request_id>/classification-runs/<run_id>/confirm	Confirm Run
POST	/portal/collection-requests/<request_id>/classification-runs/<run_id>/manual	Manual Run
GET	/notifications	List Notifications
POST	/notifications/<notification_id>/read	Mark Notification Read
POST	/notifications/read-all	Mark All Notifications Read
GET	/health/live	Live
GET	/health/ready	Ready